

CSC 212: Data Structures and Abstractions

19: Hash Tables (part 2)

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

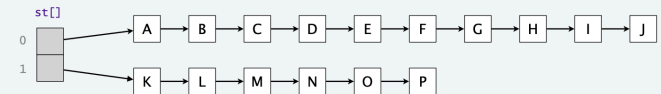
Spring 2026



Resizing a hash table

- Growing to a larger array when α exceeds a threshold
 - ✓ create a new table with larger capacity and rehash all the keys

before resizing ($n/m = 8$)



after resizing ($n/m = 4$)

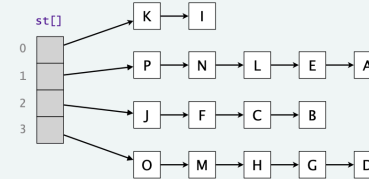


Image credit: COS 226 @ Princeton

2

Practice

- Insert the following keys into a hash of size $M=4$ (use separate chaining)
 - 4, 2, 1, 10, 21, 32, 43, 3, 51, 71
- Resize the table to $M=11$

3

Open addressing

Open addressing

Collision resolution mechanism

- ✓ all elements are stored directly within the table array, with at most one element per slot
- ✓ upon a collision, a **probe sequence** is followed until an **empty** slot is found
- ✓ duplicate keys are not permitted; requires $M \geq N$

Core operations (assume a hash function h)

- ✓ **insert**: if $h(key)$ is empty, place the new element there, otherwise, follow the probe sequence until an empty or **tombstone** slot is found
- ✓ **search**: beginning at $h(key)$, follow the probe sequence until the key is found (success), or an empty slot is reached (failure); a **tombstone** slot does not terminate the search
- ✓ **delete**: locate the key as in search, then mark the slot as deleted (**tombstone**), the slot **must not** be marked empty, as this would prematurely terminate future probe sequences and incorrectly report keys as absent

Comments

- ✓ open addressing requires no auxiliary memory for pointers or chains, making it more space-efficient than separate chaining
- ✓ at low load factors (typically $\alpha \leq 0.5$), open addressing can outperform separate chaining due to superior cache locality, as all data resides in a single contiguous array; performance degrades sharply as α approaches 1.0

5

Probing

Linear probing

- ✓ probes next available index sequentially
- ✓ $h(k, i) = (h'(k) + i) \mod m$

- m : table size
- i : probe number ($i = 0, 1, 2, \dots$)
- $h'(k)$: initial hash value of key k
- $h(k, i)$: position for the i -th probe
- $h_2(k)$: secondary hash function

Quadratic probing

- ✓ probes next available index using a quadratic function
- ✓ $h(k, i) = (h'(k) + i^2) \mod m$ **guaranteed to probe all slots only when m is prime and $\alpha < 0.5$**

Double hashing

- ✓ probes next available index using a secondary hash function h_2 (should not evaluate to 0)
- ✓ $h(k, i) = (h'(k) + i \cdot h_2(k)) \mod m$

6

Practice

Insert the following keys into a hash of size $M=13$

- 4, 2, 1, 10, 21, 32, 43, 3, 51, 71, 17

✓ linear probing

✓ quadratic probing

✓ double hashing

- $h_2(k) = 1 + (k \mod 10)$

Image credit: CS106B @ Stanford

7

Data Structure	Worst-case			Average-case			Ordered?
	insert at	delete	search	insert at	delete	search	
sequential (unordered)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	No
sequential (ordered) binary search	$O(n)$	$O(n)$	$O(\log n)$	$O(n)$	$O(n)$	$O(\log n)$	Yes
BST	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes
2-3-4	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes
Red-Black	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes
Hash table (separate chaining)	$O(1)$	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$	$O(1)^*$	No
Hash table (open addressing)	$O(n)$	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$	$O(1)^*$	No

(*) assumes uniform hashing and appropriate load factor

8

Unordered associative containers (STL)

Unordered associative containers implement data structures that can be quickly searched – $O(1)$ average-case complexity

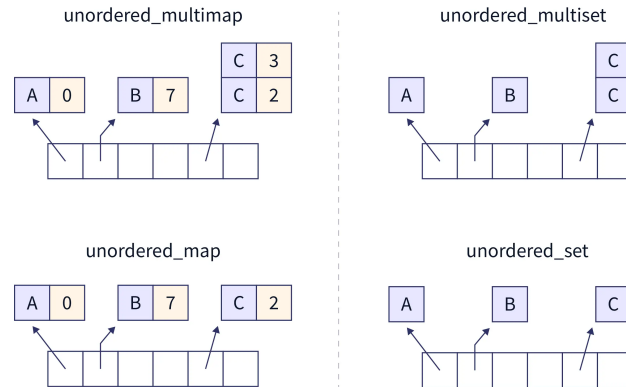


Image credit: <https://www.scaler.com/topics/cpp/containers-in-cpp/>

9

Practice

- Consider the code below that finds duplicate tokens on an input text
- modify it in a way that it finds **rare tokens** (less than k occurrences) — may need a different container

```
#include <iostream>
#include <unordered_set>
#include <string>
#include <sstream>

int main() {
    std::unordered_set<std::string> uniqueToks;
    std::unordered_set<std::string> duplicates;
    std::string line, token;

    while (std::getline(std::cin, line)) {
        std::istringstream stream(line);
        while (stream >> token) {
            if (!uniqueToks.insert(token).second) {
                duplicates.insert(token);
            }
        }
    }

    std::cout << duplicates.size() << " duplicate tokens found:\n";
    for (const auto& token : duplicates) {
        std::cout << token << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

10

```
#include <iostream>
#include <vector>
#include <set>
#include <unordered_set>
#include <string>
#include <chrono>
#include <algorithm>
#include <iomanip>

void testVector(const std::vector<std::string>& words) {
    auto start = std::chrono::high_resolution_clock::now();
    std::vector<std::string> vec;
    int duplicateCount = 0;
    for (const auto& word : words) {
        if (std::find(vec.begin(), vec.end(), word) != vec.end()) duplicateCount++;
        else vec.push_back(word);
    }
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> duration = end - start;
    std::cout << "Vector: " << duplicateCount << " duplicates, " << std::fixed << std::setprecision(6) << duration.count() << " seconds\n";
}

void testSet(const std::vector<std::string>& words) {
    auto start = std::chrono::high_resolution_clock::now();
    std::set<std::string> s;
    int duplicateCount = 0;
    for (const auto& word : words) {
        if (!s.insert(word).second)
            duplicateCount++;
    }
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> duration = end - start;
    std::cout << "Set: " << duplicateCount << " duplicates, " << std::fixed << std::setprecision(6) << duration.count() << " seconds\n";
}

void testUnorderedSet(const std::vector<std::string>& words) {
    auto start = std::chrono::high_resolution_clock::now();
    std::unordered_set<std::string> us;
    int duplicateCount = 0;
    for (const auto& word : words) {
        if (!us.insert(word).second)
            duplicateCount++;
    }
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> duration = end - start;
    std::cout << "Unordered Set: " << duplicateCount << " duplicates, " << std::fixed << std::setprecision(6) << duration.count() << " seconds\n";
}

int main() {
    std::vector<std::string> testWords;
    for (int i = 0; i < 1000000; i++)
        testWords.push_back("word" + std::to_string(i % 100000));
    std::cout << "Processing " << testWords.size() << " words...\n\n";
    testUnorderedSet(testWords);
    testSet(testWords);
    testVector(testWords);
    return 0;
}
```

11

Comparing efficiency